影像處理
--CH5_1

姓名:邱榮誠

學號:314512065

系級:電控碩 1

日期:2025/11/19

Q1.Please use Inverse Filter and Wiener Filter to correct the image corrupted severe turbulence of the assigned image, 'Fig5.25.jpg', and print out the source code and the corrected image?

1）**source code**:

```
import os
from PIL import Image, ImageDraw, ImageFont
import numpy as np
import matplotlib.pyplot as plt

IMG_PATH = r"C:\Users\邱榮誠\Desktop\DIP HW5_1\Fig5.25.jpg"
OUT_DIR = os.path.dirname(IMG_PATH) or "."
K_TUNED = 0.13
K_BORDER_EST_GUARD = 1e-12
EPS = 1e-6
KERNEL_K = 0.0025
RADIAL_RADIUS = 45
BORDER_WIDTH_FOR_K = 5

def load_gray(path):
    img = Image.open(path).convert("L")
    arr = np.asarray(img, dtype=np.float32) / 255.0
    return arr

def save_float_as_uint8(arr, path):
    a = np.clip(arr, 0.0, 1.0)
    im = Image.fromarray((a * 255.0).astype(np.uint8))
    im.save(path)

def build_turbulence_H(M, N, k=0.0025):
    u = np.arange(-M//2, M//2)
    v = np.arange(-N//2, N//2)
    U, V = np.meshgrid(v, u)
    r2 = (U.astype(np.float64)**2 + V.astype(np.float64)**2)
    H = np.exp(-k * (r2 ** (5.0/6.0)))
    return np.fft.ifftshift(H)
```

```python
def fft2(img):
    return np.fft.fft2(img)

def ifft2(spec):
    return np.fft.ifft2(spec)

def radial_mask(M, N, radius):
    u = np.arange(-M//2, M//2)
    v = np.arange(-N//2, N//2)
    U, V = np.meshgrid(v, u)
    R = np.sqrt(U**2 + V**2)
    mask = (R <= radius).astype(np.float32)
    return np.fft.ifftshift(mask)

def normalize01(x):
    x = np.array(x, dtype=np.float64)
    x = x - x.min()
    if x.max() > 0:
        x = x / x.max()
    return x

if __name__ == "__main__":
    if not os.path.exists(IMG_PATH):
        raise FileNotFoundError(f"Input image not found: {IMG_PATH}")
    os.makedirs(OUT_DIR, exist_ok=True)
    img = load_gray(IMG_PATH)
    M, N = img.shape
    print(f"Loaded image {IMG_PATH} shape={img.shape}")
    H = build_turbulence_H(M, N, k=KERNEL_K)
    G = fft2(img)
    H_safe = H.copy()
    H_safe[np.abs(H_safe) < EPS] = EPS
    F_hat_full = G / H_safe
    f_hat_full = np.real(ifft2(F_hat_full))
    mask = radial_mask(M, N, RADIAL_RADIUS)
    F_hat_limited = (G / H_safe) * mask
    f_hat_limited = np.real(ifft2(F_hat_limited))
    bw = BORDER_WIDTH_FOR_K
```

```
    top = img[:bw, :].ravel()
    bottom = img[-bw:, :].ravel()
    left = img[:, :bw].ravel()
    right = img[:, -bw:].ravel()
    border_pixels = np.concatenate([top, bottom, left, right])
    sigma_n2 = np.var(border_pixels) + 0.0
    sigma_img2 = np.var(img)
    sigma_f2 = max(sigma_img2 - sigma_n2, 1e-9)
    K_est = sigma_n2 / (sigma_f2 + K_BORDER_EST_GUARD)
    H_conj = np.conjugate(H)
    den_est = (np.abs(H)**2) + K_est
    W_est = H_conj / den_est
    F_hat_wiener_est = W_est * G
    f_hat_wiener_est = np.real(ifft2(F_hat_wiener_est))
    den_tuned = (np.abs(H)**2) + K_TUNED
    W_tuned = H_conj / den_tuned
    F_hat_wiener_tuned = W_tuned * G
    f_hat_wiener_tuned = np.real(ifft2(F_hat_wiener_tuned))
    orig_vis = normalize01(img)
    full_vis = normalize01(f_hat_full)
    limited_vis = normalize01(f_hat_limited)
    wiener_est_vis = normalize01(f_hat_wiener_est)
    wiener_tuned_vis = normalize01(f_hat_wiener_tuned)
    save_float_as_uint8(orig_vis, os.path.join(OUT_DIR,
"Fig5.25_original.png"))
    save_float_as_uint8(full_vis, os.path.join(OUT_DIR,
"Fig5.25_inverse_full.png"))
    save_float_as_uint8(limited_vis, os.path.join(OUT_DIR,
"Fig5.25_inverse_radial.png"))
    save_float_as_uint8(wiener_est_vis, os.path.join(OUT_DIR,
"Fig5.25_wiener_K_est.png"))
    save_float_as_uint8(wiener_tuned_vis, os.path.join(OUT_DIR,
"Fig5.25_wiener_K_tuned.png"))
    labels = [
        "Original (blurred)",
        "Inverse - Full",
        f"Inverse - Radial r={RADIAL_RADIUS}",
        f"Wiener K_est={K_est:.3e}",
```

```
        f"Wiener K={K_TUNED}"
    ]
    arrays = [orig_vis, full_vis, limited_vis, wiener_est_vis,
wiener_tuned_vis]

    fig, axes = plt.subplots(1, 3, figsize=(13, 4))
    show_arrays = [orig_vis, limited_vis, wiener_tuned_vis]
    show_labels = ["Original (blurred)",
                   f"Inverse - Radial r={RADIAL_RADIUS}",
                   f"Wiener K={K_TUNED}"]
    for ax, arr, title in zip(axes, show_arrays, show_labels):
        ax.imshow(arr, cmap='gray', vmin=0, vmax=1)
        ax.set_title(title, fontsize=10)
        ax.axis('off')
    plt.suptitle("Restoration Result (Only Final images)", fontsize=12)
    plt.tight_layout()
    plt.show()
    imgs = [Image.fromarray((a * 255).astype(np.uint8)).convert("L") for
a in arrays]
    spacing = 10
    w_total = sum(im.width for im in imgs) + spacing * (len(imgs) - 1)
    h_total = imgs[0].height + 24
    canvas = Image.new("L", (w_total, h_total), color=255)
    x = 0
    try:
        font = ImageFont.load_default()
    except Exception:
        font = None
    for i, im in enumerate(imgs):
        canvas.paste(im, (x, 0))
        draw = ImageDraw.Draw(canvas)
        text = labels[i]
        text_w, text_h = draw.textsize(text, font=font)
        tx = x + (im.width - text_w)//2
        ty = im.height + 6
        draw.text((tx, ty), text, fill=0, font=font)
        x += im.width + spacing
    out_comp = os.path.join(OUT_DIR, "Fig525_results_comparison.png")
```

```
    canvas.save(out_comp)
    print("Saved outputs to:", OUT_DIR)
    print(" - Fig5.25_original.png")
    print(" - Fig5.25_inverse_radial.png")
    print(" - Fig5.25_wiener_K_tuned.png")
    print(" - Fig525_results_comparison.png")
    print(f"Estimated K from border variance: {K_est:.6e}")
```

(2) corrected image:

Original img After inverse filter img After wiener filter img

Q2.Please use Inverse Filter and Wiener Filter to correct the image corrupted by motion blur of the assigned image, 'book-cover-blurred.tif', and print out the source code and the corrected image?

1)source code:

```
import numpy as np
import matplotlib.pyplot as plt
from numpy.fft import fft2, ifft2, ifftshift

IMG_PATH = "book-cover-blurred.tif"

def center_transform(img):
    M, N = img.shape
    u = np.arange(M)
    v = np.arange(N)
    X, Y = np.meshgrid(v, u)
    D = X + Y
```

```
    return img * ((-1) ** D)

def filter_H_no_shift(M, N, a, b, T):
    u = np.arange(M) - M / 2.0
    v = np.arange(N) - N / 2.0
    V_mesh, U_mesh = np.meshgrid(v, u)
    D = (V_mesh * b + U_mesh * a) * np.pi
    epsilon = 1e-12
    H = np.where(np.abs(D) < epsilon,
                 T,
                 T / D * np.sin(D))
    return H

def wiener_filter1(k, F_fft, H):
    H2 = np.abs(H) ** 2
    H_stabilized = np.where(np.abs(H) > 1e-6, H, 1e-6)
    W = (1.0 / H_stabilized) * (H2 / (H2 + k))
    return W

def read_image_and_convert_to_gray_float(filename):
    try:
        img = plt.imread(filename)
    except FileNotFoundError:
        print(f"Error: cannot open file: {filename}")
        return None
    except Exception as e:
        print(f"Error while reading image: {e}")
        return None
    if img.ndim == 3:
        img_gray = np.dot(img[..., :3], [0.2989, 0.5870, 0.1140])
    else:
        img_gray = img
    img_float = img_gray.astype(np.float64)
    if img_float.max() > 1.0:
        img_float /= 255.0
    return img_float

def main():
```

```
    fig_original = read_image_and_convert_to_gray_float(IMG_PATH)
    if fig_original is None:
        print("Failed to load book-cover-blurred image. Please check the
path.")
        return
    H_book, W_book = fig_original.shape
    print("Image size:", H_book, "x", W_book)
    F_book = fft2(center_transform(fig_original))
    a_motion = 0.1
    b_motion = 0.1
    T_motion = 1.0
    threshold_stab = 1e-6
    k_wiener_motion = 0.001
    h_no_shift = filter_H_no_shift(H_book, W_book, a_motion, b_motion,
T_motion)
    H_safe = np.where(np.abs(h_no_shift) > threshold_stab,
                      h_no_shift,
                      threshold_stab)
    inverse_restored_fft = F_book / H_safe
    inverse_restored_raw =
np.clip( np.real(ifftshift(ifft2(inverse_restored_fft))), 0, 1 )
    wiener_restored_book_fft = wiener_filter1(k_wiener_motion, F_book,
h_no_shift)
    wiener_restored_book_raw = np.clip(
        np.real(ifftshift(ifft2(wiener_restored_book_fft * F_book))), 0,
1)
    SHIFT_Y_2 = H_book // 2
    SHIFT_X_2 = W_book // 2
    inverse_restored = np.roll(np.roll(inverse_restored_raw, SHIFT_Y_2,
axis=0),SHIFT_X_2, axis=1)
    wiener_restored_book = np.roll(np.roll(wiener_restored_book_raw,
SHIFT_Y_2, axis=0),SHIFT_X_2, axis=1)
    plt.imsave("inverse_restored.png", inverse_restored, cmap="gray")
    plt.imsave("wiener_restored.png", wiener_restored_book, cmap="gray")
    plt.figure(figsize=(15, 5))
    plt.subplot(1, 3, 1)
    plt.imshow(fig_original, cmap='gray')
    plt.title('Book Cover - Blurred Image')
```

```
    plt.axis('off')
    plt.subplot(1, 3, 2)
    plt.imshow(inverse_restored, cmap='gray')
    plt.title('Inverse Filtering (shift corrected)')
    plt.axis('off')
    plt.subplot(1, 3, 3)
    plt.imshow(wiener_restored_book, cmap='gray')
    plt.title(f'Wiener Filtering (K={k_wiener_motion}, shift
corrected)')
    plt.axis('off')
    plt.suptitle('Restoration of book-cover-blurred.tif (Motion Blur)')
    plt.tight_layout()
    plt.savefig("comparison.png", dpi=150)
    plt.show()
    print("Book cover restoration finished. Saved:")
    print("  inverse_restored.png")
    print("  wiener_restored.png")
    print("  comparison.png")

if __name__ == "__main__":
    main()
```

2) **corrected image :**

Original img　　After inverse filter img　　After wiener filter img

Q3.Please comment and compare your two design filters?

在本次影像修復設計中，我針對退化模型的不同特性，分別採用了穩定化的 Inverse Filter 與考慮噪聲的 Wiener Filter。Inverse Filter 直接對退化函數取倒數，理論上能完全逆轉模糊，但因為退化函數在高頻區域通常幅值很小，若直接相除會造成噪聲嚴重放大。因此在設計上，我加入安全界線 ε 並對嚴重大氣擾動的情況額外採用低通遮罩(radial mask)，限制高頻的逆濾波範圍，藉此降低噪聲爆炸的問題。

相比之下，Wiener Filter 透過加入影像訊號與噪聲能量比的估計(K)，可同時兼顧細節修復與噪聲抑制。本作業中，我針對 Fig5.25 以影像外框方差推估噪聲並再進行 K 的調整，使得結果視覺上更銳利且不致過度增強雜訊；而在 book-cover-blurred 的圖像中，因退化方向較明確，我採用小 K 值以提升文字邊緣的可辨識性。

從上述照片來看，Inverse Filter 傾向讓細節快速增強，但也更容易受到噪聲干擾；而 Wiener Filter 則能更穩定地還原影像結構，特別是在含雜訊或嚴重模糊的情況下，視覺品質顯著優於逆濾波。因此我推論 Wiener Filter 在實際影像修復應用中更具實務價值。